Doc 5: CDQN Concept Proposal Paper - cdqnDB Architecture - Version V1.0.0

Metadata:

- **Version:** V1.0.0
- **Date:** 2025-07-14T07:15:00Z
- **Agent:** Gemini: Google (2025-07-09)
- **Lead Author:** Christophe Duy Quang Nguyen
- **Human Contributors:** N/A
- **License:** BaDaaS License - The Agile Commercial Open-Core License (Doc 2)
- **Changelog:** Initial official release of the CDQN Concept Proposal Paper focusing on cdqnDB architecture. This version incorporates corrections and ensures consistency with Doc 4 V2.0.0, including updated terminology, logical flow, and formatting of source references to align with Doc 1 specifications.
- **Summary:** This document outlines the foundational concepts of cdqnDB within the CDQN project, detailing its modular architecture as cdqnHiveDB for cloud/datacenter and cdqnBeeDB for edge devices, and explaining its crucial role in providing context and facilitating the self-adaptation and self-evolution of nano language models through integration with cdqnProtocol and cdqnLang.
- **Sections Affected:** All sections.
- **Contact:** cdqn5249@gmail.com

1. Introduction: The CDQN Vision and cdqnDB's Central Role

The CDQN (Context Datas Quorum Nodes) project is proposed as a transformative paradigm shift aiming to deliver **ubiquitous and affordable Artificial Intelligence (AI)** by leveraging the collective intelligence of numerous, highly specialized, and extremely efficient "nano agents". This vision is deeply inspired by natural systems where complex behaviors emerge from simple interactions of many individual components. Unlike the current paradigm dominated by large, centralized cloud-based Large Language Models (LLMs) that demand immense computational resources and impose significant latency, privacy, and cost barriers, CDQN seeks to enable AI operation on **low-power, embedded systems**.
The overall swarm structure of interacting nano-agents and nodes within this framework is called the **cdqNetwork**.

The core architecture of CDQN is a **decentralized multi-agent system** built upon three foundational pillars: **cdqnDB**, **cdqnProtocol**, and **cdqnLang**.

Within this ecosystem, **cdqnDB** serves as the central **evolving collective database and memory** for the entire cdqNetwork. It is designed to manage the lifecycle of data, including storage, retrieval, compression, pruning, and potential "fusion" of redundant or complementary knowledge, thereby mimicking synaptic plasticity.

This document elaborates on the modular design of cdqnDB, specifically introducing **cdqnHiveDB** for centralized, high-resource environments and **cdqnBeeDB** for distributed, edge computing settings, and details how this architecture supports the self-adaptation and self-evolution of nano LMs through its interplay with cdqnProtocol and cdqnLang.

2. cdqnDB: Architecture and Modularity (cdqnHiveDB and cdqnBeeDB)

The cdqnDB is envisioned as a new type of distributed, evolving database that serves as the collective memory and storage for the entire cdqNetwork. Its design incorporates a **modular approach** to accommodate the diverse computational and environmental requirements of the cdqNetwork, distinguishing between high-capacity central deployments and resource-constrained edge deployments. This modularity ensures that detection nodes operate collaboratively, sharing minimal metadata and making localized decisions, thus avoiding bottlenecks related to centralization.

- **cdqnHiveDB: Cloud and Datacenter Deployments**
    - **Concept:** cdqnHiveDB instances operate on **virtual servers** that aggregate resources from physical servers within data centers or cloud environments. This design is optimized for **scalability, resilience, and low latency** in cloud-native infrastructures.
    - **Underlying Technologies:** cdqnHiveDB leverages advancements in **disaggregated memory systems** and **distributed LLM serving**. For instance, **Mooncake** presents a KVCache-centric disaggregated architecture that separates prefill and decoding clusters, efficiently utilizing CPU, DRAM, SSD, and Network Interface Card (NIC) resources of GPU clusters to establish a disaggregated KV-Cache. This significantly enhances global cache capacity and inter-node transfer bandwidth. Similarly, **Beehive** is a memory-disaggregated framework that improves remote access throughput by exploiting asynchrony within threads. These technologies are crucial for the efficient "Low-Level Data Transfer Layer" of the cdqnProtocol.
    - **Orchestration:** cdqnHiveDB deployments benefit from **global orchestrators** designed for distributed LLM serving, which include capabilities for KV cache prefetching and KV-Cache-Aware Load Balancing. These orchestrators, similar to adaptive split inference frameworks, dynamically manage large foundation model (LFM) partitions across nodes to ensure real-time Quality of Service (QoS)-aware management and optimal resource utilization.
- **cdqnBeeDB: Edge Device Deployments**
    - **Concept:** cdqnBeeDB instances are designed for deployment on **edge devices** such as smartphones, tablets, personal computers, or local servers that possess sufficient resources to host them. This addresses the CDQN project's goal of enabling AI on low-power, embedded systems.
    - **Challenges and Benefits at the Edge:** Performing inference with LFMs in resource-constrained and heterogeneous edge environments presents significant challenges due to time-varying network, compute, and storage conditions. **Static split inference strategies** are often suboptimal in these dynamic settings. cdqnBeeDB addresses this by supporting **real-time, QoS-aware management** and ensuring **privacy preservation** by localizing sensitive computations directly on the edge. This localization makes reverse engineering data from model weights significantly more challenging for attackers.
    - **Enabling Technologies:** The efficiency of cdqnBeeDB is supported by **nano agents** that are inherently designed for microcontrollers, edge AI chips, and low-power CPUs. The adaptive split inference orchestration framework is critical for dynamic management of LFM partitions across edge nodes, aligning with **6G objectives** for seamless connectivity, low inference latency, and intelligent edge

resource management. Models like **Jan-nano**, a 4-billion-parameter language model, demonstrate that intelligence can be achieved through strategic design prioritizing tool integration and information retrieval, running competitively on consumer hardware.

The distributed and modular structure of cdqnDB, encompassing both cdqnHiveDB and cdqnBeeDB, is crucial for its scalability and delivers low latency, making it suitable for deployment in existing cloud security systems and diverse edge environments.

3. cdqnDB's Role in Nano LM Context, Self-Adaptation, and Self-Evolution

cdqnDB is central to providing context to nano LMs and facilitating their self-adaptation and self-evolution within the cdqNetwork. Its design is intrinsically linked to how these intelligent agents learn, reason, and improve over time.

- **Context Provisioning for Nano LMs:**
  - The core unit of storage in cdqnDB is the **Context Data Unit**, which encapsulates not just content (e.g., plaintext, sensor data, model weights, activation states, structured graphs, or prompt templates) but also **rich metadata** such as provenance, confidence scores, temporal tags, and semantic context. This detailed metadata enables the formation of a **structured and queryable knowledge graph** within the swarm.
  - **Querying Mechanisms:** cdqnDB's querying mechanisms are informed by advanced **graph-based Retrieval-Augmented Generation (RAG)** approaches like **NodeRAG** and **GraphRAG-Causal**. These methods enable **hybrid search capabilities** that combine semantic embeddings with structural cues for highly relevant and contextually aware retrieval. By providing accurate and diverse contextual information, cdqnDB mitigates **hallucination phenomena** in LLMs.
- **Enabling Self-Adaptation and Self-Evolution:**
  - **Reinforcement Learning (RL):** RL will be the primary mechanism for nano agents to self-evolve and continually improve at their tasks. Agents are trained to generate their own finetuning data and update directives, with their downstream performance serving as the reward signal for the RL loop. The **SEAL framework**, for instance, uses RL to generate more optimal finetuning data, which could be stored and managed within cdqnDB. This approach aligns with models like **MiniMax-M1** that are trained using large-scale RL on diverse problems, including real-world software engineering environments.
  - **Inverse Reasoning:** Inspired by **SAGE-nano**, cdqnDB supports nano agents in introspecting their own reasoning processes through **inverse reasoning**. This novel paradigm allows LLMs to decompose and explain their reasoning chains post-hoc, providing metacognitive self-reflection and insight into *why* specific reasoning choices were made, thus enhancing interpretability alongside reasoning performance. This self-reflection can inform subsequent data generation and model updates stored in cdqnDB.
  - **Adaptive Compilation and Feedback Loops:** The cdqnLang's "Adaptive Compilation" or "living compiler" dynamically optimizes nano agent code based on the deployment environment and observed agent behavior, using feedback loops. This process is akin to **ALTA**'s compiler, which maps programs to Transformer

weights, and **AuDaLa**'s ability to provide practical compilation for fast parallel execution on various hardware. The constant feedback and refinement cycles, which can be stored and analyzed in cdqnDB, enable continuous improvement. Compiler feedback and guided hints are used in frameworks like **UniTranslator** to refine translations iteratively, reducing bugs.

Through this synergistic integration, cdqnDB acts as the dynamic knowledge backbone, enabling nano agents to not only access relevant context but also to learn, adapt, and evolve autonomously based on real-world interactions and internal reflections.

4. Detailed Specifications and Inter-component Relationships

The effectiveness of the cdqnDB within the CDQN framework relies on its intricate design and its seamless interaction with cdqnProtocol and cdqnLang.

- **Detailed Specifications of cdqnDB:**
  - **Core Storage Unit:** The cdqnDB's core unit of storage is the **Context Data Unit**, a reproduction and extension of the **MemCube** concept from the MemOS framework. A Context Data Unit encapsulates diverse content, including plaintext, sensor data, model weights, activation states, structured graphs, or prompt templates.
  - **Rich Metadata:** Crucially, each Context Data Unit also includes **rich metadata**. This metadata encompasses attributes such as provenance, confidence scores, temporal tags, semantic context, access permissions, versioning, last usage time, source model, usage frequency, priority, expiration dates, embedding fingerprints, and storage mode. This comprehensive metadata is fundamental for structured and queryable **knowledge graph formation** within the swarm.
  - **FSSF Sub-database Layers:** cdqnDB introduces **FSSF (Factual, Semi-factual, Semi-fiction, and Fiction) sub-database layers**. These layers are implemented as a semantic classification within the metadata of Context Data Units, leveraging flexible metadata.tags or payload.format attributes. This classification allows cdqnDB to manage different types of knowledge based on their veracity and origin, enabling **fine-grained control and retrieval based on trustworthiness** and helping to mitigate issues like hallucination. This multi-level classification is supported by MemOS's **MemGovernance** features, including access control, versioning, and provenance auditing, ensuring secure and trustworthy intelligent agent systems.
  - **Evolving Schema and Performance Optimization:** The cdqnDB's **evolving schema mimics synaptic plasticity**, allowing for dynamic restructuring, migration, and seamless transitions between different memory types: parameter memory (knowledge embedded in model weights), activation memory (contextual inference state, such as KV caches), and plaintext memory (external structured knowledge). For performance optimization, cdqnDB integrates concepts from **KV cache management systems** like **LMStack** and **Mooncake**. This includes decoupling KV caches from serving engines to provide better performance and manageability. It also implements dynamic compression, composition, and modification of KV caches through a **Knowledge Delivery Network (KDN)**.
- **Relation with cdqnProtocol (Multi-layered Agentic Communication Layer):**
  - **cdqnProtocol** defines the communication paradigms for the cdqNetwork,

facilitating efficient and intelligent inter-agent communication through a **multi-layered protocol**.
    - **Low-Level Data Transfer:** The "Low-Level Data Transfer Layer" of cdqnProtocol ensures efficient and high-throughput data movement between cdqnBeeDB and cdqnHiveDB nodes. This layer draws inspiration from technologies like **RDMA networks** used in **Mooncake** and the principles of **disaggregated memory systems** seen in **Beehive**.
    - **Bio-Inspired Primitives:** The "Bio-Inspired Primitives Layer" utilizes cdqnDB for the dynamic creation and decay of "digital pheromone trails" (e.g., pheromone_signal, SwarmSearch), guiding agent interactions and information propagation within the network.
    - **Security Countermeasures:** cdqnProtocol proactively integrates robust security and safety countermeasures throughout its layers. This includes data minimization, multi-layered input validation, output consensus filtering, sandboxing of tool execution, and continuous monitoring to defend against privacy leakage, **OverThink attacks** (excessive token consumption), and tool-related vulnerabilities. Examples of existing communication protocols, such as **Model Context Protocol (MCP)** and **Agent2Agent (A2A)**, define how agents interact and exchange context.
- **Relation with cdqnLang (Object-Context-Oriented Programming Language):**
    - **Object-Context-Oriented Paradigm:** cdqnLang is proposed as a novel programming language with an **object-context-oriented programming paradigm**.
    - **Direct cdqnDB Interaction:** Operations for interacting with cdqnDB, such as cdqnDB.publish_context_data(type, content, metadata) and cdqnDB.query(filter_criteria), are not merely API calls but are **core language features**, abstracting away complex distributed data management and making interaction with the collective memory intuitive and efficient.
    - **Codified Reasoning:** cdqnLang will facilitate **codified reasoning** by enabling nano agents to express their multi-agent interactions, planning, execution, and feedback in modular pseudocode, enriched with control structures (e.g., loops, conditionals), boolean logic, and typed variables. This approach significantly enhances the interpretability, reduces ambiguity, and improves the traceability and token efficiency of multi-step reasoning processes, similar to the **CodeAgents** framework.
    - **Adaptive Compilation:** cdqnLang will feature an **Adaptive Compilation** or "living compiler." This compiler will dynamically optimize nano agent code on the fly based on the deployment environment and observed agent behavior, using feedback loops. This is akin to **ALTA**'s compiler that maps programs to Transformer weights and **AuDaLa**'s ability to provide practical compilation for fast parallel execution on various hardware. The compiler's optimization decisions can leverage the rich context and performance data stored in cdqnDB.

This comprehensive integration ensures that cdqnDB is not merely a storage solution but a dynamic, intelligent memory system that underpins the entire CDQN architecture, driving efficient communication and self-evolving behaviors across the network.

5. Conclusion

The proposed cdqnDB architecture, with its modular distinction between cdqnHiveDB and cdqnBeeDB, is a cornerstone of the CDQN project's vision for ubiquitous and affordable AI.

By treating memory as a system-level resource and integrating advanced concepts from distributed systems, graph-based RAG, and self-evolutionary AI, cdqnDB enables nano agents to operate with unprecedented contextual awareness, adaptability, and autonomy.
Its detailed specifications, including FSSF layers and KV cache optimizations, ensure robust knowledge management and performance.
Furthermore, the symbiotic relationship with cdqnProtocol for secure and efficient communication, and cdqnLang for intuitive, context-aware programming, solidifies the foundation for a truly intelligent, self-evolving, and symbiotic AI ecosystem.

This modular and integrated design facilitates seamless integration with existing security frameworks and supports horizontal scaling in distributed data centers, laying the groundwork for future AI systems that are not confined to data centers but are an integral part of our everyday lives and critical infrastructure.

6. Glossary

- **A2A (Agent2Agent) Protocol:** A communication protocol designed for interactions between AI agents.
- **Adaptive Compilation:** A process where a compiler dynamically optimizes code based on the execution environment and observed program behavior, as envisioned for cdqnLang.
- **ALTA:** A framework for programming that allows programs to be compiled into Transformer models, demonstrating adaptive compilation.
- **AuDaLa (Autonomous Data Language):** A data-autonomous programming language where computation is performed by autonomous data elements, enabling fast parallel execution.
- **BaDaaS License:** The Agile Commercial Open-Core License, governing the use of CDQN documents.
- **Beehive:** A memory-disaggregated framework that improves remote access throughput by exploiting asynchrony within threads, relevant for cdqnProtocol's Low-Level Data Transfer Layer.
- **cdqnBeeDB:** The component of cdqnDB deployed on resource-constrained edge devices (e.g., smartphones, tablets, PCs, local servers).
- **CDQN (Context Datas Quorum Nodes) Project:** A transformative initiative aiming to deliver ubiquitous and affordable AI through specialized nano agents in a decentralized multi-agent system.
- **cdqnHiveDB:** The component of cdqnDB deployed on virtual servers in data centers or cloud environments, aggregating resources from physical servers.
- **cdqnDB:** The proposed evolving, distributed database and collective memory for the CDQN network, designed to store and manage "Context Data".
- **cdqnLang:** The proposed new object-context-oriented programming language designed for developing and managing agents within the CDQN framework.
- **cdqnProtocol:** The proposed set of communication concepts and protocols for inter-agent communication within the CDQN framework.
- **cdqNetwork:** The overall swarm structure of interacting nano-agents and nodes within the CDQN framework.
- **CodeAgents:** A prompting framework that codifies multi-agent reasoning into structured pseudocode, enhancing interpretability and token efficiency.

- **Context Data Unit:** The atomic unit of storage in cdqnDB, encapsulating content (e.g., text, sensor data) along with rich metadata. It is analogous to a MemCube.
- **FSSF (Factual, Semi-factual, Semi-fiction, Fiction):** Proposed semantic classification layers within the metadata of Context Data Units in cdqnDB, designed to manage knowledge based on veracity and origin, enhancing knowledge management and reducing hallucination risks.
- **GraphRAG-Causal:** An innovative framework that combines graph-based retrieval with large language models to enhance causal reasoning.
- **Inverse Reasoning:** A novel paradigm enabling LLMs to decompose and explain their own reasoning chains post-hoc, providing insight into why specific reasoning choices were made, as used in SAGE-nano.
- **KDN (Knowledge Delivery Network):** A system for dynamically compressing, composing, and modifying Key-Value (KV) caches to optimize storage and delivery.
- **LMStack:** A system designed to address challenges in distributed LLM deployments by decoupling the KV cache from serving engines.
- **MCP (Model Context Protocol):** A protocol designed for simplifying the integration of AI applications and agents by standardizing how LLMs access and utilize external context.
- **MemCube:** A basic unit of memory in the MemOS framework, encapsulating memory content and metadata, serving as inspiration for cdqnDB's Context Data Units.
- **MemOS:** A memory operating system that treats memory as a manageable system resource, unifying plaintext, activation-based, and parameter-level memories.
- **Mixture-of-Experts (MoE):** A neural network architecture that expands model capacity without a proportional increase in per-token compute by routing inputs to specialized expert sub-networks.
- **Mooncake:** A KVCache-centric disaggregated architecture for serving LLM chatbots, featuring efficient utilization of disaggregated resources.
- **Nano Agent:** Highly specialized and efficient AI agents designed to run on microcontrollers, edge AI chips, and low-power CPUs as part of the cdqNetwork.
- **NodeRAG:** A heterogeneous graph-centric framework that enables seamless integration of graph-based methodologies into the Retrieval-Augmented Generation (RAG) workflow.
- **Object-Context-Oriented Programming:** A proposed programming paradigm for cdqnLang where Context Data Units are treated as native, first-class objects.
- **Reinforcement Learning (RL):** A machine learning paradigm where an agent learns to make decisions by performing actions in an environment to maximize a cumulative reward, used for the self-evolution of nano agents.
- **SAGE-nano:** A 4-billion-parameter reasoning model that employs inverse reasoning for LLM self-reflection.

7. References (excluding CDQN Documents)

- Alhamadi, N. (2025). Distributed Denial of Service (DDoS) Flooding Attack Mitigation Using Ensemble Deep Learning and Multi-Agent Systems for Cloud Computing Environments. *Journal of Soft Computing and Data Mining*, *6*(1), 406–422.
- Cheng, Y. (n.d.). *Decoupling KV Cache from LLMs: Towards Efficient Distributed LLM Serving*. (Doctoral dissertation).
- Cheng, Y. (n.d.). *Knowledge Delivery Network*. (Doctoral dissertation).
- Cheng, Y. (n.d.). *Global Orchestrator for Distributed LLM Serving*. (Doctoral dissertation).
- Dao, A., & Vu, D. B. (2025). Jan-nano Technical Report. arXiv preprint arXiv:2506.22760.

- Franken, T. T. P., Neele, T., & Groote, J. F. (2025). The Autonomous Data Language – Concepts, Design and Formal Verification. arXiv preprint arXiv:2506.19457.
- Google Cloud. (n.d.). *Vertex AI Agent Engine Memory Bank overview | Generative AI on Vertex AI | Google Cloud*. Retrieved from https://cloud.google.com/vertex-ai/docs/agent-engine/memory-bank
- Han, R., et al. (2024). NodeRAG: Structuring Graph-based RAG with Heterogeneous Nodes. OpenReview.
- Hu, Y., et al. (2025). MemOS: A Memory Operating System for Large Language Models. arXiv preprint arXiv:2507.03724.
- Jha, B., Paudel, F., Puri, U., Yuting, Z., Donghyuk, C., & Junhao, W. (2025). Thinking About Thinking: SAGE-nano's Inverse Reasoning for Self-Aware Language Models. arXiv preprint arXiv:2507.00092.
- Kalra, A., Rawat, A. S., Rawat, H., Haq, S., Joshi, T. T., Trivedi, H., Vardhamanan, S., Tian, Y., & Zaheer, M. (2024). A Comprehensive Survey of Mixture of Experts. arXiv preprint arXiv:2408.0583.
- Karafili, E. (2024). A RAG-based Question-Answering Solution for Cyber-Attack Investigation and Attribution. arXiv preprint arXiv:2408.06272.
- Karanjai, R., Blackshear, S., Xu, L., & Shi, W. (2025). COLLABORATION IS ALL YOU NEED: LLM ASSISTED SAFE CODE TRANSLATION. arXiv preprint arXiv:2503.11237.
- Koch, F., Djuhera, A., & Binotto, A. (2025). Intelligent Orchestration of Distributed Large Foundation Model Inference at the Edge. arXiv preprint arXiv:2504.03668.
- Kong, D., Lin, S., Xu, Z., Wang, Z., Li, M., Li, Y., Zhang, Y., Peng, H., Sha, Z., Li, Y., Lin, C., Wang, X., Liu, X., Zhang, N., Chen, C., Khan, M. K., & Han, M. (2025). A Survey of LLM-Driven AI Agent Communication: Protocols, Security Risks, and Defense Countermeasures. arXiv preprint arXiv:2506.19676.
- Li, Q., et al. (2025). Beehive: Accelerating Disaggregated Memory with Compiler-Managed Asynchrony in Multithreaded Programs. In *Proceedings of NSDI 2025*.
- Li, Q., Shi, X., & Li, R. (2025). Model Context Protocol (MCP): A Context-Oriented Protocol for LLM Agents. arXiv preprint arXiv:2504.16736.
- Madaan, A., et al. (2023). Self-Refine: Iterative refinement with self-feedback. arXiv preprint arXiv:2303.17651.
- MiniMax. (2025). MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention. arXiv preprint arXiv:2506.13585.
- Mol, D. R. J. (n.d.). *PRIVACY-PRESERVING, ENERGY-AWARE, AND LATENCY-OPTIMIZED ARCHITECTURE INTEGRATING 6G, EDGE INTELLIGENCE, AND CONTEXT-AWARE NETWORKING*.
- Qin, R., et al. (2025). Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies*.
- Shaw, P., Cohan, J., Eisenstein, J., Lee, K., Berant, J., & Toutanova, K. (2024). ALTA: Compiler-Based Analysis of Transformers. arXiv preprint arXiv:2410.18077.
- Ullah, I., Abbas, A., & Babar, M. (2025). GraphRAG-Causal: Enhancing Causal Reasoning in News Analysis with Graph-Based Retrieval and Large Language Models. arXiv preprint arXiv:2506.11600.
- Yang, B., He, X., Gao, H., Cao, Y., Li, X., & Hsu, D. (2025). CodeAgents: A Token-Efficient Framework for Codified Multi-Agent Reasoning in LLMs. arXiv preprint arXiv:2507.03254.